



Profesor: Gerardo Ramírez Villarreal

Seguridad en el Desarrollo de Aplicaciones

Práctica 1

Alumna:

Valeria Hernández Cortés



IDGS17

Tabla comparativa de las 10 vulnerabilidades OWASP

Categoría OWASP	Descripción Corta	Ejemplo de Ataque / Escenario	Impacto Potencial	Métodos de Mitigación
A01:2025 Control de acceso averiado	Garantiza que los usuarios no actúen fuera de sus permisos. Los fallos permiten realizar funciones empresariales fuera de sus límites, así como divulgar, modificar o destruir datos.	Un atacante modifica el parámetro 'acct' en el navegador para enviar cualquier número de cuenta y acceder a ella si no se verifica.	Divulgación no autorizada de información, modificación o destrucción de todos los datos, o realización de funciones fuera de los límites del usuario.	• Denegar por defecto, excepto para recursos públicos. • Implementar controles de acceso una sola vez y reutilizarlos, minimizando el uso de CORS. • Utilizar herramientas o patrones establecidos con controles sencillos y declarativos.
A02:2025 Configuración de seguridad incorrecta	Ocurre cuando un sistema, aplicación o servicio en la nube se configura de forma errónea desde el punto de vista de la seguridad, creando vulnerabilidades.	En una consola de administración no se modifican las cuentas predeterminadas; el atacante inicia sesión con la contraseña por defecto y toma el control.	Exposición de información confidencial, fallos subyacentes (como versiones de componentes vulnerables) o acceso a datos guardados en la nube.	• Crear un proceso de endurecimiento repetible para un despliegue rápido y seguro. • Usar una plataforma minimalista sin funciones, componentes, documentación ni ejemplos innecesarios. • Implementar un proceso automatizado para verificar las configuraciones en todos los entornos.

A03:2025 Fallos en la cadena de suministro de software	Interrupciones o deficiencias en la creación, distribución o actualización de software, causadas por vulnerabilidades o cambios maliciosos en código o dependencias de terceros.	El robo de 1,500 millones de dólares de Bybit en 2025, provocado por un ataque a la cadena de suministro en un software de billetera ejecutado al usar la billetera objetivo.	Los componentes suelen correr con los mismos privilegios que la aplicación, por lo que un fallo genera un impacto grave. (Ej: "Log4Shell" CVE-2021-44228 causó ransomware y criptominería).	• Inventariar continuamente versiones de componentes (cliente/servidor) y dependencias con herramientas como OWASP Dependency Track, Dependency Check o retire.js. • Obtener componentes solo de fuentes oficiales y seguras. • Actualizar periódicamente CI/CD, IDE y herramientas de desarrollo. • Reforzar sistemas habilitando MFA y restringiendo IAM.
A04:2025 Fallos criptográficos	Se centra en debilidades por falta de criptografía, uso de criptografía insuficientemente robusta, filtración de claves criptográficas y errores relacionados.	Un atacante monitorea el tráfico de red inalámbrica insegura, degrada conexiones de HTTPS a HTTP, intercepta solicitudes, roba la cookie de sesión y la reutiliza para secuestrar la sesión.	• Exposición de hashes sin sal mediante tablas arcoíris. • Descifrado por GPU de hashes generados por funciones simples o rápidas, incluso si se les añadió sal.	• Cifrar todos los datos confidenciales en reposo. • Cifrar datos en tránsito únicamente con protocolos superiores o iguales a TLS 1.2. • Almacenar contraseñas con funciones hash robustas, adaptativas y con sal (Argon2, yescrypt, scrypt o PBKDF2-HMAC-SHA-512). • Utilizar cifrado autenticado en lugar de cifrado simple.

A05:2025 Inyección	Fallo que permite enviar datos de entrada de usuario no confiables a un intérprete (navegador, base de datos o línea de comandos), provocando que ejecute esa entrada como comandos.	Un atacante modifica el parámetro 'id' enviando, lo que cambia la consulta para devolver todos los registros de la tabla.	• Acceso no autorizado a datos cuando la entrada se concatena en consultas. • Modificación o eliminación de datos, o invocación de procedimientos almacenados en ataques más peligrosos.	• Mantener los datos separados de los comandos y consultas. • Usar APIs seguras que eviten el intérprete, proporcionen interfaces parametrizadas o migrar a herramientas ORM. • Usar validación positiva de entradas del lado del servidor. • Escapar caracteres especiales con la sintaxis específica del intérprete.
A06:2025 Diseño insecure	Categoría amplia que representa debilidades expresadas como «diseño de control inexistente o ineficaz». No se puede corregir con una implementación perfecta porque los controles nunca se crearon.	Los atacantes modelan el flujo de negocio y prueban vectores de ataque, como reservar seiscientos asientos en todos los cines a la vez en pocas solicitudes, causando pérdidas masivas.	• Pérdida masiva de ingresos. • Pésima publicidad para los fabricantes de tarjetas de video y dueños de cadenas de tiendas, generando resentimiento con los entusiastas que no pueden adquirirlas.	• Establecer un ciclo de vida de desarrollo seguro con profesionales de AppSec para evaluar y diseñar controles. • Usar modelado de amenazas en partes críticas (autenticación, control de acceso, lógica de negocio). • Integrar comprobaciones de plausibilidad en cada capa (frontend a backend). • Segregar inquilinos de forma robusta por diseño en todas las capas.

A07:2025 Fallos de autenticación	Vulnerabilidad presente cuando un atacante logra engañar a un sistema para que reconozca a un usuario inválido o incorrecto como un usuario legítimo.	Si una víctima cree cerrar sesión correctamente pero sigue autenticada en algunas aplicaciones, un atacante puede usar el mismo navegador posteriormente y acceder a la cuenta.	• Uso de la aplicación como un oráculo de contraseñas para validar credenciales y obtener acceso no autorizado. • Acceso directo a la cuenta de la víctima.	• Implementar y exigir autenticación multifactor (MFA) contra relleno de credenciales, fuerza bruta y reutilización de credenciales. • No implementar el sistema con credenciales predeterminadas (menos para administradores). • Limitar o retrasar progresivamente los intentos fallidos de inicio de sesión. • Usar un administrador de sesiones seguro en el servidor que genere IDs aleatorios con alta entropía tras loguearse.
A08:2025 Fallos en la integridad del software o de los datos	Relacionados con código e infraestructura que no protegen contra el uso de código o datos no válidos/no confiables, tratándolos erróneamente como si fueran seguros.	Un atacante detecta la firma del objeto Java "rO0" (en base64) y utiliza el escáner de deserialización de Java para obtener la ejecución remota de código en el servidor.	• Los atacantes podrían cargar sus propias actualizaciones para distribuirlas y ejecutarlas en todas las instalaciones. • Introducción de paquetes con código malicioso.	• Utilizar firmas digitales o mecanismos similares para verificar que el software o los datos no han sido alterados y vienen de la fuente esperada. • Garantizar que las bibliotecas y dependencias (npm, Maven) solo consuman repositorios de confianza. • Contar con un proceso de revisión para cambios de código y configuración. • Asegurar que la canalización de CI/CD tenga segregación, configuración y control de acceso adecuados.

A09:2025 Fallos en el registro y las alertas de seguridad	Sin registro y monitoreo, los ataques no se detectan; sin alertas, es muy difícil responder de manera rápida y efectiva durante un incidente de seguridad.	El operador del sitio de un proveedor de planes de salud infantil no detectó una brecha por falta de monitoreo. Un tercero les informó que un atacante modificó miles de registros médicos de 3.5 millones de niños.	La brecha de datos podría haber estado activa y en curso desde el año 2013, es decir, durante más de siete años.	• Asegurar que los fallos de inicio de sesión, control de acceso y validación de entradas en el servidor se registren con suficiente contexto de usuario. • Retener los registros el tiempo necesario para permitir análisis forenses posteriores.
A10:2025 Gestión inadecuada de situaciones excepcionales	Ocurre cuando los programas no previenen, detectan ni responden a situaciones inusuales e impredecibles, provocando fallos, comportamientos inesperados o vulnerabilidades.	Exposición de datos confidenciales mediante errores de la base de datos que revelan el error completo del sistema al usuario. El atacante provoca más errores para perfeccionar una inyección SQL.	Este tipo de vulnerabilidades pueden afectar negativamente la confidencialidad, la disponibilidad y/o la integridad de un sistema o de sus datos.	• Incluir una validación estricta de la entrada (con saneamiento o escape de caracteres peligrosos que se deban aceptar). • Implementar un manejo centralizado de errores, registro, monitoreo, alertas y un gestor global de excepciones.

Casos reales por vulnerabilidad

A01:2025 - Control de acceso roto



Caso 1: Facebook y la exposición de fotos privadas

Nombre de la vulnerabilidad: Exposición de fotos privadas / Brecha de seguridad por exportación de tokens de acceso

Empresa afectada: Facebook

Fecha: en el año 2018

Cómo fue explotada: La vulnerabilidad se

originó a partir de la función concebida para que los usuarios pudieran visualizar cómo sus perfiles son vistos por otras personas. Los atacantes idearon una forma de exportar “tokens de acceso”, que utilizaron para obtener el control de las cuentas de otros usuarios.

Referencia: Panda Security (Ene 9, 2019).
<https://www.pandasecurity.com/es/mediacenter/facebook-2018-cronologia-ano/>

Caso 2: Zoom, cuando reuniones privadas accesibles sin autenticación

Nombre de la vulnerabilidad: Reuniones privadas accesibles sin autenticación / Falta de limitación de velocidad

Empresa afectada: Zoom

Fecha: 2020 (con reportes del 03/04/2020, 1 de abril de 2020, y solución el 9 de abril de 2020)

Cómo fue explotada: Almacenaba los videos guardados sin una contraseña y bajo el mismo nombre, por lo que una simple búsqueda en línea permitía encontrarlos. Asimismo, en el cliente web existía una falta de limitación de velocidad que permitió que un atacante, mediante un script de prueba de concepto basado en Python, intentara todas las 1 millón de combinaciones de contraseñas numéricas de seis dígitos en cuestión de minutos (ataque de fuerza bruta) enviando continuamente solicitudes HTTP para probar todas las combinaciones y obtener acceso a las reuniones privadas. También se descubrió que podía repetirse contra una lista de los 10 millones de contraseñas principales en reuniones programadas con variantes alfanuméricas.

Referencia: Ariadna Cruz / Techbit (03/04/2020).
<https://www.eluniversal.com.mx/techbit/se-filtran-en-internet-videollamadas-hechas-por-zoom/> y Editor Contenidos / Blog AMECI (05 Agosto 2020).
<https://www.ameci.org/blog/error-en-zoom-permitia-obtener-el-pin-privado-de-las-reuniones>

A02:2025 - Error de configuración de seguridad

Caso 1: Yahoo con múltiples brechas por cifrado deficiente



Nombre de la vulnerabilidad: Múltiples brechas por cifrado deficiente / Campaña de spearphishing y cookies falsificadas

Empresa afectada: Yahoo

Fecha: Ocurrieron en agosto de 2013 y finales de 2014 (reveladas públicamente en septiembre de 2016, diciembre de 2016 y octubre de 2017)

Cómo fue explotada: Ambas brechas explotaron sistemas de seguridad inadecuados y métodos de cifrado débiles como el hashing MD5. Los atacantes crearon cookies falsificadas y usaron scripts maliciosos para generar accesos no autorizados y obtener acceso persistente a cuentas de usuario sin contraseñas. La brecha de 2013 implicó el uso de código malicioso para eludir los controles de cuentas, mientras que la de 2014 utilizó una campaña de spearphishing para robar credenciales administrativas y filtrar grandes cantidades de datos sensibles.

Referencia: Tutoriales Programación Ya.
<https://www.tutorialesprogramacionya.com/historiasoftware/ataquesenciberseguridad/tema27.html>
y Huntress Threat Library.
<https://www.huntress.com/threat-library/data-breach/yahoo-data-breach>

Caso 2: LinkedIn con contraseñas almacenadas con SHA-1

Nombre de la vulnerabilidad: Contraseñas almacenadas con SHA-1 / Aplicación móvil que enviaba datos sin cifrar

Empresa afectada: LinkedIn

Fecha: Jueves 7 junio 2012

Cómo fue explotada: Los atacantes robaron archivos que contenían contraseñas encriptadas únicamente con SHA-1. Posteriormente, un usuario ('DWDMM') solicitó ayuda en un foro ruso de hackers para descifrarlas y obtener las contraseñas originales. En un incidente separado, una aplicación móvil enviaba sin cifrar a los servidores los números de teléfono y contraseñas para llamadas en conferencia.

Referencia: Autor ASI (jueves 7 junio 2012).
<https://blog.auditoria.com.mx/2012/06/contrasenas-de-linkedin-expuestas/> y SinEmbargo MX (18/05/2016).
<https://www.sinembargo.mx/1662037/linkedin-revela-que-el-hackeo-en-2012-expuso-117-millones-de-contrasenas/>

A03:2025 - Fallos en la cadena de suministro de software

Caso 1: Tesla, bug bounty reportó SQL Injection

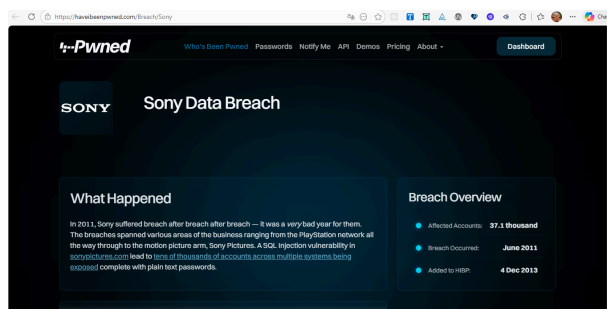
Nombre de la vulnerabilidad: Inyección SQL (SQL Injection) en el lado del servidor que llevó a la ejecución remota de código por un problema de deserialización (CVE-2020-0618)

Empresa afectada: Tesla (en un servidor para socios que alojaba Microsoft SQL Server Reporting Services)

Fecha: Reportado el 15 de febrero de 2014

Cómo fue explotada: El investigador ('Parzel') buscó dominios Tesla que alojaran el servicio Microsoft SQL Server Reporting Services (SSRS) sin parchear utilizando cadenas de código fuente como huellas dactilares. Los servidores SSRS vulnerables tenían problemas para gestionar solicitudes de página especialmente diseñadas generando un problema de deserialización. El atacante explotó esto enviando una solicitud HTTP con una carga útil generada en PowerShell para obtener un *reverse shell* (ejecución remota de código) requiriendo únicamente estar autenticado con privilegios mínimos.

Referencia: Ionut Ilascu / BleepingComputer (February 20, 2020). <https://www.bleepingcomputer.com/news/security/tesla-pays-10k-for-microsoft-sql-server-reporting-services-bug/>



Caso 2: Sony Pictures y el SQL Injection masivo

Nombre de la vulnerabilidad: Vulnerabilidad de SQL Injection masivo

Empresa afectada: Sony Pictures (división cinematográfica de Sony)

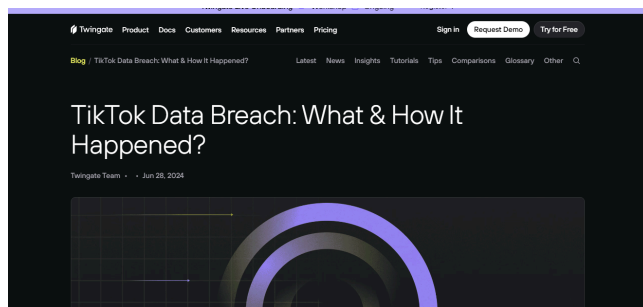
Fecha: Junio de 2011

Cómo fue explotada: Una vulnerabilidad de SQL Injection en el sitio web

sonypictures.com provocó que decenas de miles de cuentas en múltiples sistemas fueran expuestas, dejando las contraseñas expuestas en texto plano.

Referencia: Have I Been Pwned (4 Dec 2013). <https://haveibeenpwned.com/Breach/Sony> y Asfand Khan / Kiuwan Blog (May 20th, 2021). <https://www.kiuwan.com/blog/biggest-cloud-breaches-of-2020/>

A04:2025 - Fallos criptográficos



Caso 1: TikTok y la API expuesta sin controles

Nombre de la vulnerabilidad: API expuesta sin controles / Base de datos no segura y enlaces maliciosos por mensajes privados

Empresa afectada: TikTok

Fecha: Jueves 9 de enero de 2020 /

Agosto de 2020

Cómo fue explotada: En agosto de 2020, se accedió a una base de datos no segura recopilada originalmente por Deep Social que contenía datos de cuentas de redes sociales y era accesible en internet sin contraseña ni cifrado. En junio de 2024, los hackers explotaron las vulnerabilidades de TikTok enviando enlaces maliciosos a través de mensajes privados para secuestrar cuentas destacadas, incluida la de CNN.

Referencia: Twingate Team (Jun 28, 2024).
<https://www.twingate.com/blog/tips/TikTok-data-breach>

Caso 2: WhatsApp con cifrado mal diseñado

Nombre de la vulnerabilidad: Cifrado inicial mal diseñado / Envío de mensajes en texto plano

Empresa afectada: WhatsApp

Fecha: 2011 (con incidentes derivados en 2012, 2014 y 2016)

Cómo fue explotada: La aplicación enviaba los mensajes utilizando texto plano sin algoritmos de cifrado, haciendo accesible el ciberataque o secuestro de información durante el proceso de envío. Adicionalmente, en 2012 el sitio web WhatsAppStatus.net permitía cambiar el estado de WhatsApp de cualquier usuario de manera externa debido a estas deficiencias de seguridad.

Referencia: MDZol (18 de mayo de 2026).
<https://www.mdzol.com/sociedad/2021/1/19/escandalo-en-whatsapp-que-significan-los-mensajes-encriptados-el-cifrado-de-extremo-extremo-133810.html>

A05:2025 - Inyección



Caso 1: MongoDB

Nombre de la vulnerabilidad: Miles de bases abiertas / Instalaciones expuestas sin protección por contraseña

Empresa afectada: MongoDB (múltiples bases de datos desprotegidas en internet)

Fecha: 2017 (con una ola de

ataques automatizados detectados desde abril de este año y documentados el 03 Jul 2020)

Cómo fue explotada: Un cibercriminal utilizó scripts automáticos para escanear internet y buscar instalaciones de MongoDB expuestas públicamente sin protección por contraseña. El script se infiltraba en ellas, eliminaba de manera automatizada todo su contenido y dejaba una nota de rescate exigiendo un pago en bitcoins a cambio de devolver los datos.

Referencia: Amer Owaida (03 Jul 2020).
<https://www.welivesecurity.com/la-es/2020/07/03/secuestran-informacion-bases-datos-mongodb-atacante-pide-pago-rescate/>

Caso 2: Datos expuestos de la NASA

Nombre de la vulnerabilidad: Datos expuestos en servidores públicos / Acceso no autorizado a sistemas

Empresa afectada: NASA

Fecha: Descubierto el 23 de octubre de 2018 (notificaciones enviadas en noviembre de 2018, divulgación pública en diciembre de 2018)

Cómo fue explotada: Un intruso no autorizado accedió a servidores sensibles que almacenaban información personal identificable (PII) de empleados y contratistas. Los

atacantes accedieron debido a vulnerabilidades en los sistemas de la NASA, posiblemente mediante software sin parches o el uso de credenciales filtradas.

Referencia: Lizzie Danielson / Huntress (11/14/2025).
<https://www.huntress.com/threat-library/data-breach/nasa-data-breach>

A06:2025 - Diseño inseguro



Caso 1: Autenticación insegura de Paypal

Nombre de la vulnerabilidad: Autenticación insegura en API / Spear-phishing e interceptación de comunicaciones internas

Empresa afectada: PayPal (dentro de un ataque conjunto a los sitios web de eBay Reino Unido)

Fecha: Enero de 2014 (con eventos específicos el 25-01-2014 y el 1 de febrero de 2014)

Cómo fue explotada: Hackers pro-sirios utilizaron phishing dirigido (spear-phishing) contra el personal de seguridad respondiendo al incidente, potencialmente explotando vulnerabilidades en Java o Flash a través de dispositivos comprometidos. Esto les permitió infiltrar al menos una cuenta de correo electrónico o laptop de un interviniente corporativo, logrando interceptar correos internos y escuchar de manera simultánea la coordinación de sus llamadas de conferencia sobre respuestas a incidentes para monitorizar sus estrategias de mitigación.

Referencia: CSIDB (25-01-2014).
<https://www.csidb.net/csldb/incidents/ef59f314-9928-40c4-a9f2-daddfdf980c8/>

Caso 2: Equifax

Nombre de la vulnerabilidad: Vulnerabilidad no parcheada en Apache Struts (CVE-2017-5638)

Empresa afectada: Equifax

Fecha: May-July 2017 revelada en 2017

Cómo fue explotada: Los atacantes explotaron una vulnerabilidad en Apache Struts que había recibido un parche de seguridad dos meses antes, pero que la empresa no había aplicado. Además, un certificado SSL caducado impidió que el sistema de detección de intrusos de Equifax viera la exfiltración de datos durante 76 días, lo que expuso datos financieros sensibles de casi la mitad de todos los adultos estadounidenses.

Referencia: Josh Amishav / BreachSense (Last updated Apr 13, 2026).
<https://www.breachsense.com/blog/data-breach-examples/>

A07:2025 - Fallos en la autenticación



Caso 1: Hackeo a Twitter

Nombre de la vulnerabilidad: Hackeo por autenticación débil / Compromiso de sistemas

internos por Spearphishing selectivo

Empresa afectada: Twitter

Fecha: Miércoles 15 de julio de 2020

Cómo fue explotada: Los ciberdelincuentes utilizaron un enfoque de phishing selectivo (spearphishing) dirigido en dos pasos. Primero, comprometieron e ingresaron a la cuenta de Slack de Twitter, donde supuestamente encontraron credenciales expuestas que les dieron acceso a los servidores internos de la empresa para obtener información sobre sus procesos. Con este conocimiento, atacaron a otros empleados que sí tenían acceso a las herramientas de soporte y gestión de cuentas, logrando asegurar sus privilegios para ingresar a 130 cuentas, restablecer contraseñas de 45 de ellas, publicar tuits de estafa con Bitcoin y descargar datos privados.

Referencia: Alianza de Lázaro (12 de agosto de 2020). <https://lazarusalliance.com/es/Lecciones-clave-de-ciberseguridad-extra%C3%ADdas-del-hackeo-de-Twitter/>

Caso 2: Hackeo a Instagram

Nombre de la vulnerabilidad: Bypass de autenticación / Base de datos en línea no protegida por un tercero

Empresa afectada: Instagram (asociada a la agencia de marketing Chtrbox)

Fecha: Mayo de 2019

Cómo fue explotada: La empresa de marketing de influencers Chtrbox recopiló datos detallados de perfiles de Instagram (posiblemente por acceso inadecuado a la API o extractos previos) y almacenó dicha base de datos en un servidor no seguro de Amazon Web Services (AWS), el cual quedó expuesto públicamente en internet y accesible para cualquiera sin contraseña ni controles de autenticación.

Referencia: Sam Kirkpatrick / CyberNewsLive (17 June 2025). <https://cybernewslive.com/instagram-chtrbox-data-leak-how-a-2019-breach-still-shapes-social-media-security/>

A08:2025 - Fallos de integridad de software o datos



Caso 1: Plugins alterados de wordpress

Nombre de la vulnerabilidad: Plugins alterados / Ataque a la cadena de suministro por adquisición de software inyectado con puerta trasera de ejecución remota de código (RCE)

Empresa afectada: WordPress

(afectando a usuarios del portafolio "Essential Plugin")

Fecha: Infección inicial introducida el 8 de agosto de 2025

Cómo fue explotada: Un comprador ('Kris') adquirió legítimamente el negocio de plugins en el marketplace Flippa. Tras asumir el control, subió una actualización SVN (versión 2.6.7) que añadía sigilosamente 191 líneas de código PHP malicioso que contenían un método fetch_ver_info() que pasaba datos controlados por el atacante a unserialize(), una llamada

arbitraria a funciones mediante `version_info_clean()` y un endpoint de API REST no autenticado. Tras 8 meses inactivo, el servidor de comando y control (C2) distribuyó cargas maliciosas que descargaron un archivo falso (`wp-comments-posts.php`) y lo usaron para inyectar código PHP en `wp-config.php`, exfiltrando enlaces de spam visibles únicamente para Googlebot.

Referencia: Mayura Kathir / GBHackers (April 15, 2026).
<https://gbhackers.com/trusted-wordpress-plugins/>

Caso 2: CCleaner con malware

Nombre de la vulnerabilidad: Software distribuido con malware / Binario modificado con firma digital legítima

Empresa afectada: CCleaner

Fecha: Desde agosto de 2017 hasta el 12 de septiembre de 2017

Cómo fue explotada: Los atacantes realizaron una intrusión en los sistemas de la compañía y obtuvieron acceso a sus certificados de firma digital. Utilizaron este acceso para modificar el binario legítimo del software e incluir una puerta trasera con funcionalidades de botnet, algoritmos de generación de dominios (DGA) y comunicación con un servidor Command-and-Control (C2), logrando que el software modificado fuera distribuido firmado digitalmente por la propia compañía mientras recolectaba datos del sistema de los usuarios.

Referencia: INCIBE-CERT (Fecha de publicación 20/09/2017).
<https://www.incibe.es/incibe-cert/publicaciones/bitacora-de-seguridad/popular-software-seguridad-ccleaner-modificado-distribuir>

A09:2025 - Fallos en el registro y alertas de seguridad

Caso 1: API expuesta de Toyota

Nombre de la vulnerabilidad: API expuesta por clave de acceso publicada en repositorio público de código fuente

Empresa afectada: Toyota Motor Corporation (aplicación T-Connect)

Fecha: Descubierto en septiembre de 2022 (clave expuesta desde diciembre de 2017 hasta el 15 de septiembre de 2022)

Cómo fue explotada: Un subcontratista de desarrollo publicó por error una parte del código fuente del sitio T-Connect en un repositorio público de GitHub. Dicho fragmento contenía una clave de acceso al servidor de datos que almacenaba las direcciones de correo electrónico y números de gestión de los clientes, permitiendo que cualquier tercero no autorizado tuviera acceso potencial y pudiera exfiltrar los datos de la base de datos sin restricciones durante casi cinco años.

Referencia: Noticias Seguridad (On: October 11, 2022).
<https://noticiasseguridad.com/hacking-incidentes/toyota-revela-una-fuga-de-datos-despues-de-que-la-clave-de-acceso-fuera-expuesta-en-github/>



The screenshot shows the INCIBE (Instituto Nacional de Ciberseguridad) website. The header includes the INCIBE logo and navigation links for various sectors. The main content area features a news article titled "Samsung sufre un incidente de ciberseguridad" published on 04/10/2022. The article text mentions that Samsung, a major South Korean company, has reported a security breach affecting a code source related to its Galaxy devices, but no personal information of clients or employees was compromised. It also notes that Samsung has reinforced its security measures to prevent future incidents of this type.

Caso 2: Filtración de código fuente de Samsung

Nombre de la vulnerabilidad: Filtración de código fuente sin detección temprana por falta de alertas en repositorios

Empresa afectada: Samsung

Fecha: Detectado e informado el 04/03/2022

Cómo fue explotada: El grupo de ransomware LAPSUS\$ atacó y vulneró la seguridad de la empresa, comprometiendo y filtrando un volumen de 190 GB de información que incluía código fuente relacionado con el funcionamiento de los dispositivos Galaxy. El documento proporcionado no detalla la técnica exacta de intrusión inicial en la infraestructura, resaltando la falta de alertas operativas dentro de sus repositorios de datos.

Referencia: INCIBE-CERT (Fecha de publicación 15/03/2022).
<https://www.incibe.es/incibe-cert/publicaciones/bitacora-de-seguridad/samsung-sufre-incidente-ciberseguridad>

A10:2025 - Mal manejo de condiciones excepcionales

Caso 1: Google Cloud

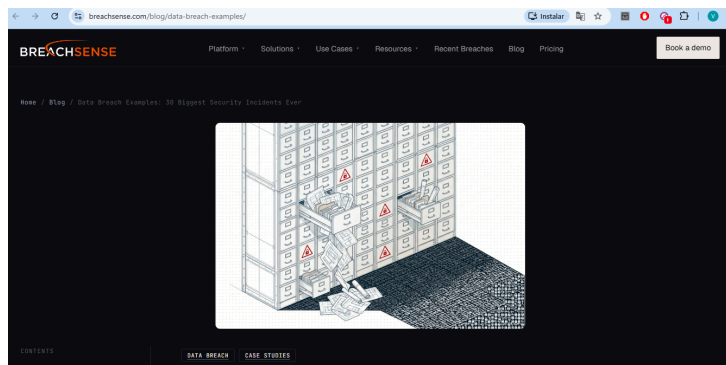
Nombre de la vulnerabilidad: SSRF en metadatos / Base de datos expuesta en la nube

Empresa afectada: Google Cloud Platform (alojando un servidor de origen o propietario no identificado)

Fecha: Enero de 2020

Cómo fue explotada: Investigadores de seguridad detectaron de forma directa en línea un host comprometido en la plataforma de Google Cloud que contenía una base de datos abierta con más de 200 millones de registros personales y demográficos de usuarios de EE. UU., expuestos de manera pública en la infraestructura de la nube.

Referencia: Asfand Khan / Kiuwan Blog (May 20th, 2021).
<https://www.kiuwan.com/blog/biggest-cloud-breaches-of-2020/>



Caso 2: T-mobile

Nombre de la vulnerabilidad: API explotada por vulnerabilidad de control de accesos / Abuso de la interfaz de programación de aplicaciones

Empresa afectada: T-Mobile

Fecha: Finales de noviembre de 2022

Cómo fue explotada: Los atacantes explotaron una vulnerabilidad de control de accesos en una de las APIs públicas de T-Mobile. Debido a la falta de un bloqueo estricto en la autenticación y a la ausencia de supervisión contra abusos de consultas, los actores maliciosos lograron realizar solicitudes continuas y extraer masivamente datos sensibles de 37 millones de clientes actuales sin ser detectados durante más de un mes.

Referencia: Josh Amishav / BreachSense (Last updated Apr 13, 2026).
<https://www.breachsense.com/blog/data-breach-examples/>

Código vulnerable

A07:2025 - Fallos en la autenticación

Código Ejemplo inseguro (contraseñas en texto plano)

```
11 import getpass
12 from typing import Dict
13
14 # ---- Demo: usuarios con contraseñas en texto plano (vulnerable) ----
15 users: Dict[str, str] = {
16     "admin": "123456", # contraseña en texto plano (insegura)
17     "user": "password"
18 }
19
20 def login(username: str, password: str) -> bool:
21     """
22     Login simple: comparación en texto plano.
23     Retorna True si coincide username+password, False en caso contrario.
24     """
25     return username in users and users[username] == password
26
27 def get_users() -> Dict[str, str]:
28     """
29     Devuelve una copia de la estructura 'users' para pruebas.
30     ADVERTENCIA: contiene contraseñas en texto plano.
31     """
32     return dict(users)
33
34 if __name__ == "__main__":
35     try:
36         username = input("Usuario: ").strip()
37         password = getpass.getpass("Contraseña: ")
38         if login(username, password):
39             print(f"¡Bienvenido {username}!")
40         else:
41             print("Usuario o contraseña incorrectos")
42     except KeyboardInterrupt:
43         print("\nInterrumpido por el usuario.")
```

Seguro # Versión segura (PBKDF2 + bloqueo + logging)

```
7 import hashlib
8 import hmac
9 import secrets
10 import time
11 import logging
12 from logging.handlers import RotatingFileHandler
13 from pathlib import Path
14 from typing import Dict
15
16 # =====
17 # CONFIGURACIÓN BASE
18 # =====
19 ITERATIONS = 200_000
20 MAX_FAILED = 3
21 LOCKOUT_SECONDS = 10
22
23 # =====
24 # LOGGING CONFIGURACIÓN
25 # =====
26 PROJECT_ROOT = Path(__file__).resolve().parents[2]
27 LOG_DIR = PROJECT_ROOT / "logs"
28 LOG_DIR.mkdir(parents=True, exist_ok=True)
29
30 LOG_FILE = LOG_DIR / "auth.log"
31 LOG_MAX_BYTES = 5 * 1024 * 1024
32 LOG_BACKUPS = 3
33
34 logger = logging.getLogger("secure_auth")
35 if not logger.handlers:
36     handler = RotatingFileHandler(LOG_FILE, maxBytes=LOG_MAX_BYTES, backupCount=LOG_BACKUPS)
37     formatter = logging.Formatter("%(asctime)s %(levelname)s %(message)s")
38     handler.setFormatter(formatter)
39     logger.addHandler(handler)
40     logger.setLevel(logging.INFO)
41
42 """
```

```
45     _USE_REDIS = False
46     _USE_DB = False
47     db_lockout = None
48     rds = None
49
50     try:
51         import redis
52         rds = redis.Redis(host="redis", port=6379, decode_responses=True)
53         rds.ping()
54         _USE_REDIS = True
55         logger.info("Redis detectado y habilitado para persistencia de lockout")
56     except Exception as e:
57         logger.warning("Redis no disponible: %s", e)
58
59     if not _USE_REDIS:
60         try:
61             from code.secure import db_lockout
62             _USE_DB = True
63             logger.info("db_lockout habilitado via paquete")
64         except Exception:
65             pass
66
67     # =====
68     # USUARIOS CON HASH Y SALT
69     # =====
70     def _hash_password(password: str, salt: bytes) -> bytes:
71         return hashlib.pbkdf2_hmac("sha256", password.encode(), salt, ITERATIONS)
72
73     _users: Dict[str, Dict[str, bytes]] = {}
74     _admin_salt = secrets.token_bytes(16)
75     _users["admin"] = {"salt": _admin_salt, "hash": _hash_password("123456", _admin_salt)}
76     _user_salt = secrets.token_bytes(16)
77     _users["user"] = {"salt": _user_salt, "hash": _hash_password("password", _user_salt)}
78
```

```
82     _failed_attempts: Dict[str, int] = {}
83     _lockout_until: Dict[str, float] = {}
84
85     def _get_status(username: str):
86         if _USE_REDIS:
87             try:
88                 failed = int(rds.get(f"auth:{username}:failed") or 0)
89                 lock_until = float(rds.get(f"auth:{username}:lock_until") or 0.0)
90                 return failed, lock_until
91             except Exception:
92                 logger.exception("Error leyendo estado desde Redis")
93         elif _USE_DB:
94             try:
95                 return db_lockout.get_status(username)
96             except Exception:
97                 logger.exception("Error en db_lockout.get_status")
98         return _failed_attempts.get(username, 0), _lockout_until.get(username, 0.0)
99
100     def _update_status(username: str, failed_count: int, lock_until: float):
101         if _USE_REDIS:
102             try:
103                 rds.set(f"auth:{username}:failed", failed_count)
104                 rds.set(f"auth:{username}:lock_until", lock_until)
105                 return
106             except Exception:
107                 logger.exception("Error actualizando estado en Redis")
108         elif _USE_DB:
109             try:
110                 db_lockout.update_status(username, failed_count, lock_until)
111                 return
112             except Exception:
113                 logger.exception("Error actualizando en db_lockout")
114         _failed_attempts[username] = failed_count
115         _lockout_until[username] = lock_until
116
```

```
110
117  def _reset_user(username: str):
118      if _USE_REDIS:
119          try:
120              rds.delete(f"auth:{username}:failed", f"auth:{username}:lock_until")
121              return
122          except Exception:
123              logger.exception("Error eliminando estado de Redis")
124      elif _USE_DB:
125          try:
126              db_lockout.reset_user(username)
127              return
128          except Exception:
129              logger.exception("Error en db_lockout.reset_user")
130      _failed_attempts.pop(username, None)
131      _lockout_until.pop(username, None)
132
133      # =====
134      # LÓGICA DE LOGIN
135      # =====
136  def login(username: str, password: str) -> bool:
137      now = time.time()
138      failed_count, lock_until = _get_status(username)
139
140      if lock_until and now < lock_until:
141          logger.info("Cuenta '%s' bloqueada hasta %s", username, lock_until)
142          return False
143
144      if username not in _users:
145          failed_count += 1
146          logger.warning("Intento con usuario no existente '%s'", username)
147          if failed_count >= MAX_FAILED:
148              new_lock = time.time() + LOCKOUT_SECONDS
149              _update_status(username, 0, new_lock)
150              logger.warning("Bloqueo aplicado a '%s' hasta %s", username, new_lock)
151          else:
```

Referencia: https://github.com/Herly123/insecure_design_lab

A01:2025 - Control de acceso roto

Código Inseguro

```
const express = require("express");
const app = express();
const { getUserData } = require("../authService");

app.get('/profile/:id', (req, res) => {
  const userId = req.params.id;
  const userData = getUserData(userId);
  res.json(userData);
});
```

Seguro


```

1  const express = require("express");
2  const app = express();
3  const { getUserData, verifyToken, checkPermissions } = require("../authService");
4
5  app.get("/profile/:id", async (req, res) => {
6    try {
7      // Verificar que se proporcione el token "Verificar": Unknown word.
8      const token = req.headers.authorization;
9      if (!token) {
10         return res.status(401).json({ message: "Acceso no autorizado" }); "Acceso": Unknown word.
11      }
12
13      const decodedUser = verifyToken(token); // Validar el token "Validar": Unknown word.
14      if (!decodedUser) {
15         return res.status(403).json({ message: "Token inválido o expirado" }); "inválido": Unknown word.
16      }
17
18      // Validar el ID del usuario como un string seguro "Validar": Unknown word.
19      const userId = req.params.id.trim(); // Eliminar espacios en blanco "Eliminar": Unknown word.
20
21      if (userId || typeof userId !== "string" || userId.length > 50 || /^[^a-zA-Z0-9-_]/.test(userId)) {
22         return res.status(400).json({ message: "ID inválido" }); "inválido": Unknown word.
23      }
24
25      // Obtener los datos del usuario "Obtener": Unknown word.
26      const userData = await getUserData(userId);
27      if (!userData) {
28         return res.status(404).json({ message: "Usuario no encontrado" }); "Usuario": Unknown word.
29      }
30
31      // Verificar autorización el usuario solo puede acceder a su propio perfil "Verificar": Unknown word.
32      if (!checkPermissions(decodedUser, userId)) {
33         return res.status(403).json({ message: "Acceso denegado" }); "Acceso": Unknown word.
34      }
35
36      // Filtrar datos sensibles antes de enviarlos "Filtrar": Unknown word.
37      const { password, token: _, ...safeUserData } = userData;
38
39      res.json(safeUserData);
40    } catch (error) {
41      res.status(500).json({ message: "Error interno del servidor" }); "interno": Unknown word.
42    }
43  });

```

Referencia: <https://www.linkedin.com/pulse/a012021-broken-access-control-entendiedo-y-mitigando-jhon-alexander-pj9ve>

A05:2025 - Inyección

Código seguro vs inseguro

Código Vulnerable	Código Seguro
<pre> Private Sub cmdBuscar_Click If Trim(txtBusqueda.Text) = = " Then Exit Sub Me.Filter = ConstSQLID & & " & txtBusqueda.Text End Sub </pre>	<pre> Private Sub cmdBuscar_Click If Trim(txtBusqueda.Text) = = " Then Exit Sub Dim filtro As String filtro = Replace(txtBusqueda.Text, "'", "'\"") Me.Filter = ConstSQLID & "=? Me.Recordset.Parameters(0) = filtro Me.Recordset.Requery End Sub </pre>

Referencia: https://www.accessdemo.info/docs_seguridad/inyeccion-sql.html

Ranking Personal

1. A02:2025 - Error de configuración de seguridad

Justificación: Los datos sensibles de los usuarios son el activo más valioso y confidencial de cualquier organización; si una plataforma expone o no sabe proteger adecuadamente esta información, pierde de inmediato toda la confianza del público, destruyendo su reputación.

2. A01:2025 - Control de acceso roto

Justificación: Los límites de permisos dentro de un sistema son la primera línea de defensa de los datos; permitir que cualquier usuario no autorizado manipule privilegios o modifique información crítica sin restricciones representa un peligro sistémico inaceptable.

3. A07:2025 - Fallos en la autenticación

Justificación: De nada sirve estructurar y programar todo un portal de inicio de sesión si los mecanismos de validación de identidad son vulnerables y pueden ser burlados fácilmente por los atacantes, dejando las cuentas completamente desprotegidas.

4. A05:2025 - Inyección

Justificación: Cuando las solicitudes maliciosas logran infiltrar y exponer la estructura o los datos internos de la base de datos, la integridad de toda la aplicación se rompe por completo, demostrando que ningún control perimetral fue útil.

5. A06:2025 - Diseño inseguro

Justificación: Esta categoría representa una debilidad vital porque los errores se originan desde la arquitectura misma de la app; aunque no estén a simple vista y el atacante deba buscarlos con esfuerzo, un diseño defectuoso condena al sistema a ser inherentemente vulnerable.

Conclusión Final

Como desarrolladora, conocer el OWASP Top 10 es fundamental porque te abre los ojos a los riesgos reales de internet. Normalmente pienso en la seguridad desde el principio, en la planeación, porque no hacerlo te lleva a tener que hacer desde cero todos y cada uno de los procedimientos. Planificar la seguridad desde el primer código te ayuda a evitar el trabajo doble, algo que lamentablemente ya me ha pasado y no dan ganas de repetir.

Lo que más me preocupa al programar una app es que salga algo mal con mi top personal: la exposición de los datos. No tener la capacidad de proteger la información de la gente, aunque yo crea que sí lo estoy haciendo bien, es un riesgo enorme. Como usuaria entiendo que la privacidad es vital; por eso, al programar, se siente como si cargara con la confianza de las personas en mis manos.

En mi práctica profesional, aplico este conocimiento usando herramientas seguras en el día a día. Esto significa programar utilizando ORMs para blindar las bases de datos contra inyecciones, configurar de manera segura las cookies y estructurar un uso correcto de sesiones utilizando tokens y JWT para que nadie pueda saltarse el inicio de sesión. Además, me comprometo a mantenerme informada constantemente para saber cómo se van actualizando las amenazas. Si una vulnerabilidad es descubierta, se extiende rápido como un virus y cualquiera puede usarla; es literalmente una enfermedad tecnológica que puede afectar a cada sitio, y la información a tiempo es la única cura.

Declaración de Uso de IA

Uso 1: ChatGPT como mi tutor: Utilizado para ayudarme a verificar y hacer más fuerte la comprensión del Top 10 de OWASP. Se interactuó con la herramienta presentándole definiciones en mis propias palabras para validar si el concepto se había entendido correctamente.

Uso 2: ChatGPT y Gemini para verificaciones: Con ChatGPT se sometieron a revisión distintos casos de la vida real encontrados en la red para verificar si correspondían a la vulnerabilidad de OWASP que yo pensaba, continuando con mi búsqueda cuando no coincidían. Con Gemini se verificó la calidad final del entregable, utilizándola para revisar la ortografía, pulir la coherencia de las ideas (organización) y verificar el formato correcto de las referencias bibliográficas.

Uso 3: Copilot y Gemini como un punto de partida :

Cuando no encontraban casos de alguna OWASP en específico, se solicitó a Copilot un listado inicial de 10 casos de la vida real donde se vulneran los estándares de esa OWASP. A partir de esa lista, se investigaron de forma independiente las fuentes para seleccionar y profundizar en los casos más interesantes.

Se utilizó a Gemini como punto de partida para la conclusión, empleando una serie de preguntas guía para estructurar y redactar con calidad mi reflexión final de manera independiente.